

Intro to R, Part I

A brief introduction to R packages

Packages are curated collections of functions and/or compiled code for use in R. Some commonly-downloaded, popular packages include sample data and documentation to explain and demonstrate implementation.

Packages can be downloaded from the Comprehensive R Archive Network (CRAN), the primary repository for R packages. Each package submitted to CRAN is reviewed and tested thoroughly before release. Every package is released with a version number, and the most popular packages are frequently updated and released with a new version number.

CRAN is a trusted source for up-to-date, consistent versions of R packages, and allows package accessibility across multiple servers in multiple regions called ‘mirrors’. Packages can also be downloaded through GitHub, but we’ll get to that later.

1. Using `install.packages()`

One of the most useful packages in R is `tidyverse`.

Install `tidyverse` by typing `install.packages('tidyverse')` in the code chunk below.

The `install.packages()` function is the easiest method to download packages from CRAN.

View the documentation for `install.packages()` by clicking [here](#).

```
install.packages("tidyverse")
```

2. The `library()` function

Once the package is downloaded, the package must be called into virtual memory before it is used. In order to use the package, call it into virtual memory with the `library()` command.

When downloading packages, the package name must be in quotation marks. When called into virtual memory with the `library()` command, no quotation marks are needed.

Put the `tidyverse` package into virtual memory by typing `library(tidyverse)` in the code chunk below.

```
library(tidyverse)
```

3. Viewing package version with `packageVersion()`

In an R Markdown document, you can begin a new code chunk by adding three tick marks followed by a lower case `r` enclosed in curly brackets: ````{r}`

You can close the code chunk with three tick marks: `````.

Create a code chunk below and find the package version of `tidyverse` you’ve just downloaded by using the `packageVersion()` function.

View the documentation for `packageVersion()` by clicking [here](#).

```
packageVersion("tidyverse")
```

```
## [1] '2.0.0'
```

4. Importing a .csv file with read.csv

Read the .csv file titled `brfss2024tri.csv` into a dataframe with the `read.csv()` function. Name the resulting dataframe `brtri`.

Hint: include the file path when reading the file, and remember to either double backslashes (`\`) or use forward slashes (`/`) in the file path.

View the documentation for `read.csv()` by clicking [here](#).

```
brtri <- read.csv("brfss2024tri.csv")
```

5. The colnames() function

View the column names in the `brtri` dataframe with the `colnames()` function in the code chunk below.

How many columns are in the dataframe?

View the documentation for `colnames()` by clicking [here](#).

View additional methods for retrieving column names and details by clicking [here](#).

```
colnames(brtri)
```

```
## [1] "X"          "STATE"      "FMONTH"     "IDATE"      "IMONTH"     "IDAY"
## [7] "IYEAR"      "DISPCODE"   "SEQNO"      "PSU"        "CTELENM1"   "PVTRES1"
## [13] "COLGHOUS"   "STATERE1"   "CELFON1"    "LADULT1"    "NUMADULT"   "RESPSLC1"
## [19] "LANDSEX3"   "SAFETIME"   "CTELNUM1"   "CELLFON5"   "CADULT1"    "CELLSEX3"
## [25] "PVTRES1"    "CCLGHOUS"   "CSTATE1"    "LANDLINE"   "HHADULT"    "SEXVAR"
## [31] "GENHLTH"    "PHYSHLTH"   "MENTHLTH"   "POORHLTH"   "PRIMINS2"   "PERSDOC3"
## [37] "MEDCOST1"   "CHECKUP1"   "EXERANY2"   "LASTDEN4"   "RMVTETH4"   "CVDINFR4"
## [43] "CVDCHRD4"   "CVDSTRK3"   "ASTHMA3"    "ASTHNO"     "CHCSCNC1"   "CHCOCNC1"
## [49] "CHCCOPD3"   "ADDEPEV3"   "CHCKDNY2"   "HAVARTH4"   "DIABETE4"   "DIABAGE4"
## [55] "MARITAL"    "EDUCA"      "RENTHOM1"   "NUMHHOL4"   "NUMPHON4"   "CPDEMO1C"
## [61] "VETERAN3"   "EMPLOY1"    "CHILDREN"   "INCOME3"    "PREGNANT"   "WEIGHT2"
## [67] "HEIGHT3"    "DEAF"       "BLIND"      "DECIDE"     "DIFFWALK"   "DIFFDRES"
## [73] "DIFFALON"   "HADMMAM"    "HOWLONG"    "CERVSCRN"   "CRVCLCNC"   "CRVCLPAP"
## [79] "CRVCLHPV"   "HADHYST2"   "HADSIGM4"   "COLNSIGM"   "COLNTES1"   "SIGMTES1"
## [85] "LASTSIG4"   "COLNLCNC"   "VIRCOL01"   "VCLNTES2"   "SMALSTOL"   "STOLTEST"
## [91] "STOOLDN2"   "BLDSTFIT"   "SDNATES1"   "SMOKE100"   "SMOKDAY2"   "USENOW3"
## [97] "ECIGNOW3"   "LCSFIRST"   "LCSNUMCG"   "LCSCTSC1"   "LCSSCNCR"   "LCSCTWHN"
## [103] "ALCDAY4"    "AVEDRNK4"   "DRNK3GE5"   "MAXDRNKS"   "FLUSHOT7"   "FLSHTMY3"
## [109] "IMFVPLA5"   "PNEUVAC4"   "HIVTST7"    "HIVTSTD3"   "HIVRISK5"   "PDIABTS1"
## [115] "PREDIAB2"   "DIABTYPE"   "INSULIN1"   "CHKHEMO3"   "EYEEEXAM1"   "DIABEYE1"
## [121] "DIABEDU1"   "FEETSORE"   "ARTHEXER"   "SHINGLE2"    "HPVADVC4"   "HPVADSH1"
## [127] "TETANUS1"   "CNCRDIF"    "CNCRAGE"    "CNCRTYP2"   "CSRVRT3"    "CSRVD0C1"
## [133] "CSRVSUM"    "CSRVRTRN"   "CSRVINST"   "CSRVINSR"   "CSRVDEIN"   "CSRVCLIN"
## [139] "CSRVPAIN"   "CSRVCTL2"   "PSATEST1"   "PSATIME1"   "PCPSARS2"   "PSASUGS1"
## [145] "PCSTALK2"   "CIMEMLO1"   "CDWORRY"    "CDDISCU1"   "CDHOUS1"    "CDSOCIA1"
## [151] "CAREGIV1"   "CRGVREL5"   "CRGVPRB4"   "CRGVALZD"   "CRGVNURS"   "CRGVPER2"
## [157] "CRGVHOU2"   "CRGVHRS2"   "CRGVNLG2"   "ACEDEPRS"   "ACEDRINK"   "ACEDRUGS"
## [163] "ACEPRISN"   "ACEDIVRC"   "ACEPUNCH"   "ACEHURT1"   "ACESWEAR"   "ACETOUGH"
## [169] "ACETHEM"    "ACEHVSEX"   "ACEADSAF"   "ACEADNED"   "LSATISFY"   "EMTSUPRT"
```

```
## [175] "SDLONELY" "SDHEMPY" "FOODSTMP" "SDHFOOD1" "SDHBILLS" "SDHUTILS"
## [181] "SDHTRNSP" "HOWSAFE1" "MARIJAN1" "MARJSMOK" "MARJEAT" "MARJVAPE"
## [187] "MARJDAB" "MARJOTHR" "USEMRJN4" "LASTSMK2" "STOPSMK2" "MENTCIGS"
## [193] "MENTECIG" "HEATTBCO" "SSBSUGR2" "SSBFRUT3" "FIREARM5" "GUNLOAD"
## [199] "LOADULK2" "RCSGEND1" "RCSXBRTH" "RCSRLTN2" "CASTHDX2" "CASTHNO2"
## [205] "SOMALE" "SOFEMALE" "HADSEX" "PFPPRVN4" "TYPCNTR9" "NOBCUSE8"
## [211] "QSTVER" "QSTLANG" "HPVDSHT" "ICFQSTVR" "METSTAT" "URBSTAT"
## [217] "MSCODE" "STSTR" "STRWT" "RAWRAKE" "WT2RAKE" "IMPRACE"
## [223] "CHISPNC" "CRACE1" "CAGEG" "CLLCPWT" "DUALUSE" "DUALCOR"
## [229] "LLCPWT2" "LLCPWT" "RFHLTH" "PHYS14D" "MENT14D" "HLTHPL2"
## [235] "HCVU654" "TOTINDA" "EXTETH3" "ALTETH3" "DENVST3" "MICHD"
## [241] "LTASTH1" "CASTHM1" "ASTHMS1" "DRDXAR2" "MRACE1" "HISPANC"
## [247] "RACE" "RACEG21" "RACEGR3" "RACEPRV" "SEX" "AGEG5YR"
## [253] "AGE65YR" "AGE80" "AGEG" "HTIN4" "HTM4" "WTKG3"
## [259] "BMI5" "BMI5CAT" "RFBMI5" "CHLDCNT" "EDUCAG" "INCOMG1"
## [265] "RFMAM23" "MAM402Y" "CRVSCRN" "RFPAP37" "HPV5YR1" "PAPHPV1"
## [271] "HADCOLN" "CLNSCP2" "HADSIGM" "SGMSCP2" "SGMS102" "RFBLDS6"
## [277] "STOLDN2" "VIRCOL2" "SBONTI2" "CRCREC3" "SMOKER3" "RFSMOK3"
## [283] "CURECI3" "LCSLAST" "LCSNUMC" "LCSAGE" "LCSYSMK" "PACKDAY"
## [289] "PACKYRS" "LCSYQTS" "LCSSMKG" "LCSELIG" "LCSCTSN" "LCSPSTF"
## [295] "DRNKANY6" "DROCDY4" "RFBING6" "DRNKWK3" "RFDRHV9" "FLSHOT7"
## [301] "PNEUMO3" "AIDTST4"
```

6. Using `sapply()` to view column type

View the class of each column in the `brtri` dataframe by implementing the `sapply()` function with the `class()` function in the code chunk below.

View the documentation for `sapply()` by clicking [here](#).

```
sapply(brtri, class)
```

```
##          X          STATE      FMONTH      IDATE      IMONTH      IDAY      IYEAR  DISPCODE
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##      SEQNO          PSU  CTELENM1  PVTRES1  COLGHOUS  STATERE1  CELPHON1  LADULT1
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  NUMADULT  RESPSLC1  LANDSEX3  SAFETIME  CTELNUM1  CELLFON5  CADULT1  CELLSEX3
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  PVTRES3  CCLGHOUS  CSTATE1  LANDLINE  HHADULT  SEXVAR  GENHLTH  PHYSHLTH
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  MENTHLTH  POORHLTH  PRIMINS2  PERSDOC3  MEDCOST1  CHECKUP1  EXERANY2  LASTDEN4
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  RMVTETH4  CVDINFR4  CVDGRHD4  CVDSTRK3  ASTHMA3  ASTHNOW  CHCSCNC1  CHCOCNC1
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  CHCCOPD3  ADDEPEV3  CHCKDNY2  HAVARTH4  DIABETE4  DIABAGE4  MARITAL  EDUCA
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  RENTHOM1  NUMHHOL4  NUMPHON4  CPDEMO1C  VETERAN3  EMPLOY1  CHILDREN  INCOME3
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  PREGNANT  WEIGHT2  HEIGHT3  DEAF  BLIND  DECIDE  DIFFWALK  DIFFDRES
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  DIFFALON  HADMAM  HOWLONG  CERVSCRN  CRVCLCNC  CRVCLPAP  CRVCLHPV  HADHYST2
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  HADSIGM4  COLNSIGM  COLNTES1  SIGMTES1  LASTSIG4  COLNCNCR  VIRCOLO1  VCLNTES2
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
##  SMALSTOL  STOLTEST  STOOLDN2  BLDSTFIT  SDNATES1  SMOKE100  SMOKDAY2  USENOW3
## "integer" "integer" "integer" "integer" "integer" "integer" "integer" "integer"
```

##	ECIGNOW3	LCSFIRST	LCSNUMCG	LCSCTSC1	LCSSCNCR	LCSCTWHN	ALCDAY4	AVEDRNK4
##	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"
##	DRNK3GE5	MAXDRNK5	FLUSHOT7	FLSHTMY3	IMFVPLA5	PNEUVAC4	HIVTST7	HIVTSTD3
##	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"
##	HIVRISK5	PDIABTS1	PREDIAB2	DIABTYPE	INSULIN1	CHKHEMO3	EYEEXAM1	DIABEYE1
##	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"
##	DIABEDU1	FEETSORE	ARTHEXER	SHINGLE2	HPVADVC4	HPVADSH1	TETANUS1	CNCRDIFF
##	"integer"	"integer"	"integer"	"logical"	"logical"	"logical"	"logical"	"integer"
##	CNCRAGE	CNCRTYP2	CSRVTRT3	CSRVDOC1	CSRVSUM	CSRVTRTN	CSRVINST	CSRVINSR
##	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"
##	CSRVDEIN	CSRVCLIN	CSRVPAIN	CSRVCTL2	PSATEST1	PSATIME1	PCPSARS2	PSASUGS1
##	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"
##	PCSTALK2	CIMEMLO1	CDWORRY	CDDISCU1	CDHOUS1	CDSOCIA1	CAREGIV1	CRGVREL5
##	"integer"	"logical"	"logical"	"logical"	"logical"	"logical"	"integer"	"integer"
##	CRGVPRB4	CRGVALZD	CRGVNURS	CRGVPER2	CRGVHOU2	CRGVHRS2	CRGVLNG2	ACEDEPRS
##	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"logical"
##	ACEDRINK	ACEDRUGS	ACEPRISN	ACEDIVRC	ACEPUNCH	ACEHURT1	ACESWEAR	ACETOUCH
##	"logical"	"logical"	"logical"	"logical"	"logical"	"logical"	"logical"	"logical"
##	ACETTHEM	ACEHVSEX	ACEADSAF	ACEADNED	LSATISFY	EMTSUPRT	SDLONELY	SDHEMPLY
##	"logical"	"logical"	"logical"	"logical"	"integer"	"integer"	"integer"	"integer"
##	FOODSTMP	SDHFOOD1	SDHBILLS	SDHUTILS	SDHTRNSP	HOWSAFE1	MARIJAN1	MARJSMOK
##	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"
##	MARJEAT	MARJVAPE	MARJDAB	MARJOTHR	USEMRJN4	LASTSMK2	STOPSMK2	MENTCIGS
##	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"
##	MENTECIG	HEATTBCO	SSBSUGR2	SSBFRUT3	FIREARM5	GUNLOAD	LOADULK2	RCSGEND1
##	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"logical"
##	RCSXBRTH	RCSRLTN2	CASTHDX2	CASTHNO2	SOMALE	SOFEMALE	HADSEX	PFPPRVN4
##	"logical"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"
##	TYPCTR9	NOBCUSE8	QSTVER	QSTLANG	HPVDSHT	ICFQSTVR	METSTAT	URBSTAT
##	"integer"	"integer"	"integer"	"integer"	"logical"	"integer"	"integer"	"integer"
##	MSCODE	STSTR	STRWT	RAWRAKE	WT2RAKE	IMPRACE	CHISPNC	CRACE1
##	"integer"	"integer"	"numeric"	"numeric"	"numeric"	"integer"	"integer"	"integer"
##	CAGEG	CLLCPWT	DUALUSE	DUALCOR	LLCPWT2	LLCPWT	RFHLTH	PHYS14D
##	"integer"	"numeric"	"integer"	"numeric"	"numeric"	"numeric"	"integer"	"integer"
##	MENT14D	HLTHPL2	HCVU654	TOTINDA	EXTETH3	ALTETH3	DENVST3	MICHD
##	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"
##	LTASTH1	CASTHM1	ASTHMS1	DRDXAR2	MRACE1	HISPANC	RACE	RACEG21
##	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"
##	RACEGR3	RACEPRV	SEX	AGEG5YR	AGE65YR	AGE80	AGEG	HTIN4
##	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"
##	HTM4	WTKG3	BMI5	BMI5CAT	RFBMI5	CHLDCNT	EDUCAG	INCOMG1
##	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"
##	RFMAM23	MAM402Y	CRVSCRN	RFPAP37	HPV5YR1	PAPHPV1	HADCOLN	CLNSCP2
##	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"
##	HADSIGM	SGMSCP2	SGMS102	RFBLDS6	STOLDN2	VIRCOL2	SBONTI2	CRCREC3
##	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"
##	SMOKER3	RFSMOK3	CURECI3	LCSLAST	LCSNUMC	LCSAGE	LCSYSMK	PACKDAY
##	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"
##	PACKYRS	LCSYQTS	LCSSMKG	LCSELIG	LCSCTSN	LCSPSTF	DRNKANY6	DROCDY4
##	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"
##	RFBING6	DRNKWK3	RFDRHV9	FLSHOT7	PNEUMO3	AIDTST4		
##	"integer"	"integer"	"integer"	"integer"	"integer"	"integer"		

7. The `table()` function

Use the `table()` function to view the frequencies of the `STATE` column in the `brtri` dataframe in the code chunk below. Use the parameter `useNA` to view the number of rows with missing data. View the documentation for `table()` by clicking [here](#).

```
table(brtri[, "STATE"], useNA = "ifany")
```

```
##  
##      9      34      36  
## 7213 8666 43913
```

8. Viewing the number of rows with `nrow()`

Use the `nrow()` function to view the number of rows in the `brtri` dataframe in the code chunk below. View the documentation for `nrow()` by clicking [here](#).

```
nrow(brtri)
```

```
## [1] 59792
```

9. The BRFSS Codebook: Height

The Behavioral Risk Factor Surveillance Survey (BRFSS) data in the `brtri` dataframe have a codebook with the following information:

Label: Reported Height in Feet and Inches

Section Name: Demographics

Type of Variable: Num

Variable Name: HEIGHT3

Question: About how tall are you without shoes? (If respondent answers in metrics, put a 9 in the first column)[Round fractions down.]

Value	Value Label	Frequency	Percentage	Weighted Percentage
200-711	Height (ft/inches)			
	Notes: 0 __ / __ __ = feet / inches	425,837	95.85	94.10
7777	Don't know/Not sure	4,198	0.94	1.49
9061-9998	Height (meters/centimeters)			
	Notes: The initial '9' indicates this was a metric value.	7,793	1.75	2.91
9999	Refused	6,442	1.45	1.50
BLANK	Not asked or Missing	13,400		

Use the `table()` function to get a frequency table of the data in the `HEIGHT3` column in the code chunk below. Given the information in the table above, what range of values would you say are in feet and inches in column `HEIGHT3`?

What range of values would you say are in centimeters?

View the documentation for `table()` by clicking [here](#).

```
table(brtri$HEIGHT3, useNA = "ifany")
```

```
##
## 300 305 307 309 311 400 401 402 403 404 405 406 407 408 409 410
## 6 4 1 1 1 22 3 7 3 3 36 15 21 58 118 210
## 411 500 501 502 503 504 505 506 507 508 509 510 511 600 601 602
## 603 1931 1961 3659 3681 4611 4349 4841 4553 4103 3773 4189 3213 3497 1661 1410
## 603 604 605 606 607 608 609 610 611 700 701 702 704 705 706 708
## 861 495 180 79 41 27 11 12 9 5 2 2 4 3 1 1
## 711 7777 9090 9092 9095 9096 9098 9100 9102 9103 9104 9105 9106 9107 9108 9110
## 3 668 4 1 1 1 1 2 1 2 2 13 5 2 1 4
## 9111 9115 9118 9123 9125 9127 9129 9130 9132 9133 9135 9136 9137 9138 9140 9141
## 1 5 1 1 1 1 1 9 1 1 4 2 2 3 21 2
## 9142 9143 9144 9145 9146 9147 9148 9149 9150 9151 9152 9153 9154 9155 9156 9157
## 3 3 5 33 4 5 15 14 111 14 59 43 36 96 49 37
## 9158 9159 9160 9161 9162 9163 9164 9165 9166 9167 9168 9169 9170 9171 9172 9173
## 48 21 169 12 48 41 29 119 19 38 53 38 129 22 34 32
## 9174 9175 9176 9177 9178 9179 9180 9181 9182 9183 9184 9185 9186 9187 9188 9190
## 18 85 23 14 25 9 45 2 11 11 8 15 5 3 2 4
## 9191 9192 9193 9195 9196 9198 9200 9207 9251 9252 9999 <NA>
## 2 3 1 1 1 1 1 1 1 1 1 995 2081
```

10. Extracting height: feet, inches

According to the information in the BRFSS codebook, values from 200 to 711 in the `HEIGHT3` column represent height in feet and inches.

For example, the value ‘511’ actually represents a height of 5 feet and 11 inches.

The codebook also explains that values beginning with the number ‘9’ in the `HEIGHT3` column precede digits that represent height recorded in centimeters.

The number ‘9’ only signifies that the remaining digits are recorded in centimeters and should not be included in the measurement values.

In order to compare height of each respondent, we will need to convert all height values to inches or centimeters.

We will accomplish this step by step: 1. Where height is recorded in feet and inches, for example ‘511’, we’ll separate values of feet and values of inches into separate `HEIGHTft` and `HEIGHTin` columns. 2. Where height is measured in centimeters, for example ‘9170’, we will need to extract the last three digits—the values following the leading ‘9’—and store as values of centimeters in a separate `HEIGHTcm` column. 3. After we have three new columns (`HEIGHTft`, `HEIGHTin`, `HEIGHTcm`), we can convert values conditionally by row until each respondent has height recorded in inches and centimeters.

Extract the ‘foot’ (the first digit) and ‘inch’ (the last two digits) components of each respondent’s height from the `HEIGHT3` column conditionally—only if `brtri$HEIGHT3 >= 200 & brtri$HEIGHT3 <= 711`.

Use conditional methods with logical indexing and the `substr()` function to create `brtri$HEIGHTft` and `brtri$HEIGHTin` columns and populate with the correct digits from the `HEIGHT3` column.

When subsetting columns, NA values are not allowed, so remember to include `!is.na(brtri$HEIGHT3)` in the logical indexing.

You can use Stack Overflow as a resource on conditional methods with logical indexing by clicking [here](#).

View the documentation for `substr()` by clicking [here](#).

```
brtri$HEIGHTft[brtri$HEIGHT3 >= 200 & brtri$HEIGHT3 <=
  711 & !is.na(brtri$HEIGHT3)] <- as.numeric(substr(as.character(brtri$HEIGHT3[brtri$HEIGHT3 >=
  200 & brtri$HEIGHT3 <= 711 & !is.na(brtri$HEIGHT3)]),
  1, 1))
brtri$HEIGHTin[brtri$HEIGHT3 >= 200 & brtri$HEIGHT3 <=
  711 & !is.na(brtri$HEIGHT3)] <- as.numeric(substr(as.character(brtri$HEIGHT3[brtri$HEIGHT3 >=
```

```
200 & brtri$HEIGHT3 <= 711 & !is.na(brtri$HEIGHT3)]),
2, 3))
```

11. Transforming height: total inches

Create a new column called `HEIGHTtotin` and populate it with values in total inches using column operations and the new `HEIGHTft` and `HEIGHTin` columns we just created.

You should be able to create the column and populated it with values in a single line of code.

```
brtri$HEIGHTtotin <- (brtri$HEIGHTft * 12) + brtri$HEIGHTin
```

12. Extracting height: centimeters

Create a new column called `HEIGHTcm` and populate it with NA values.

Extract the height of each respondent recorded in centimeters from the `HEIGHT3` column conditionally—only if `brtri$HEIGHT3 >= 9061 & brtri$HEIGHT3 <= 9998`.

Use conditional methods with logical indexing and the `substr()` function to populate with the correct digits from the `HEIGHT3` column.

Remember that the last three digits of the `HEIGHT3` column represent respondent height in centimeters if the leading digit is a ‘9’.

When subsetting columns, NA values are not allowed, so remember to include `!is.na(brtri$HEIGHT3)` in the logical indexing.

```
brtri$HEIGHTcm <- NA
brtri$HEIGHTcm[brtri$HEIGHT3 >= 9061 & brtri$HEIGHT3 <=
9998 & !is.na(brtri$HEIGHT3)] <- as.numeric(substr(as.character(brtri$HEIGHT3[brtri$HEIGHT3 >=
9061 & brtri$HEIGHT3 <= 9998 & !is.na(brtri$HEIGHT3)]),
2, 4))
```

13. Transforming height: total centimeters

Create a new column called `HEIGHTtotcm` by simply copying the column `brtri$HEIGHTcm`.

This new column `HEIGHTtotcm` will hold the height of each respondent in centimeters.

For rows with height recorded in inches, we will convert from inches to centimeters and populate `HEIGHTtotcm` with converted values.

Use the `round()` function to round to the nearest whole centimeter.

View the documentation for `round()` by clicking [here](#).

```
brtri$HEIGHTtotcm <- brtri$HEIGHTcm
brtri$HEIGHTtotcm[is.na(brtri$HEIGHTcm) & !is.na(brtri$HEIGHTtotin)] <- round(brtri$HEIGHTtotin[is.na(brtri$HEIGHTcm) & !is.na(brtri$HEIGHTtotin)] * 2.54, 0)
```

14. Transforming height: total meters

Create a new column called `HEIGHTtotm` by simply converting the column `brtri$HEIGHTtotcm` to meters.

Use the `round()` function to round values to four decimal places.

```
brtri$HEIGHTtotm <- round(brtri$HEIGHTtotcm/100, 4)
```

15. Transforming height: total centimeters to total inches

Create a new column called `HEIGHTtotin` by simply converting the column `brtri$HEIGHTtotcm` to inches. Use the `round()` function to round values to the nearest whole inch.

```
brtri$HEIGHTtotin[!is.na(brtri$HEIGHTtotcm) & is.na(brtri$HEIGHTtotin)] <- round(brtri$HEIGHTtotcm[!is.na(brtri$HEIGHTtotcm)] * 0.393701, 0)
```

16. Respondent Sex: Creating a factor

The Behavioral Risk Factor Surveillance Survey (BRFSS) data in the `brtri` dataframe have a codebook with the following information:

Label: Sex of Respondent

Section Name: Respondent Sex

Type of Variable: Num

Variable Name: SEXVAR

Question: Sex of Respondent

Value	Value Label	Frequency	Percentage	Weighted Percentage
1	Male: Code=1 if LANDSEX3=1 or CELLSEX3=1	217,487	47.52	49.12
2	Female: Code=2 if LANDSEX3=2 or CELLSEX3=2	240,183	52.48	50.88

Create a new variable `SEX` and populate it with empty values (“”).

Use conditional logic and indexing to code the value of `SEX` for all rows where `SEXVAR==1` as ‘Male’.

Use conditional logic and indexing to code the value of `SEX` for all rows where `SEXVAR==2` as ‘Female’.

Use the `factor()` function to convert `brtri$SEX` to a factor.

View the documentation for `factor()` by clicking [here](#).

```
brtri$SEX <- ""
brtri$SEX[brtri$SEXVAR == 1] <- "Male"
brtri$SEX[brtri$SEXVAR == 2] <- "Female"
brtri$SEX <- factor(brtri$SEX, level = c("Male", "Female"))
```

17. Plotting an histogram of (unweighted) height stratified by sex

Use the code chunk below to plot a histogram of respondent height in meters stratified by sex using `ggplot` and `geom_histogram`.

Use the resources in the hyperlinks below.

In particular, view the code in the R code gallery.

View the documentation for `ggplot()` by clicking [here](#).

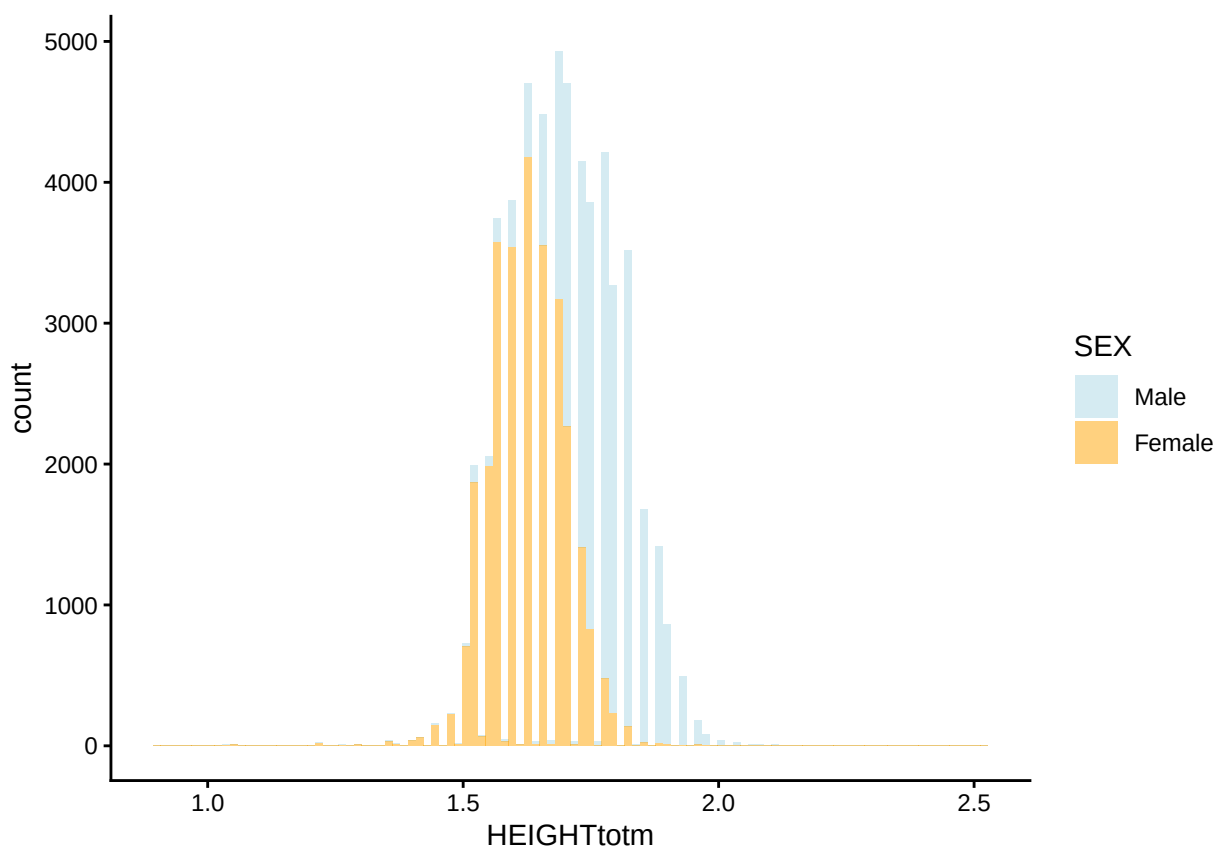
View the documentation for `geom_histogram()` by clicking [here](#).

View a gallery of R code and histograms with several groups by clicking [here](#).


```
nbins <- length(unique(brtri$HEIGHTtotm))

brtri %>%
  ggplot(aes(HEIGHTtotm, fill = SEX)) + geom_histogram(aes(fill = SEX),
    bins = nbins, alpha = 0.5) + scale_fill_manual(values = c("lightblue",
    "orange"), aesthetics = "fill", breaks = waiver(),
    na.value = "grey50") + theme_classic() + theme(legend.position = "right")
```

```
## Warning: Removed 3744 rows containing non-finite outside the scale range
## ('stat_bin()').
```



18. Compare our calculated HEIGHTtotin column values to BRFSS height column values in inches

Use the `head()` function to view the first rows of the `HEIGHTtotin` and `HTIN4` columns in the `brtri` dataframe. View the documentation for `head()` by clicking [here](#).

```
head(brtri[, c("HEIGHTtotin", "HTIN4")])
```

```
##   HEIGHTtotin HTIN4
## 1         64    64
## 2         NA    NA
## 3         67    67
## 4         67    67
## 5         64    64
## 6         60    60
```

19. Compare our calculated HEIGHTtotm column values to BRFSS height column values in meters

Use the `head()` function to view the first rows of the `HEIGHTtotm` and `HTM4` columns in the `brtri` dataframe.

```
head(brtri[, c("HEIGHTtotm", "HTM4")])
```

```
##   HEIGHTtotm HTM4
## 1      1.63  163
## 2      NA   NA
## 3      1.70  170
## 4      1.70  170
## 5      1.63  163
## 6      1.52  152
```

20. The BRFSS Codebook: Weight

The Behavioral Risk Factor Surveillance Survey (BRFSS) data in the `brtri` dataframe have a codebook with the following information:

Label: Reported Weight in Pounds

Section Name: Demographics

Type of Variable: Num

Variable Name: `WEIGHT2`

Question: About how much do you weigh without shoes? (If respondent answers in metrics, put a 9 in the first column)[Round fractions up.]

Value	Value Label	Frequency	Percentage	Weighted Percentage
50-0776	Weight (pounds)			
	Notes: 0 _ _ _ = weight in pounds	417,106	93.66	93.36
7777	Don't know/Not sure	7,740	1.74	2.11
9023-9352	Weight (kilograms)			
	Notes: The initial '9' indicates this was a metric value.	4,190	0.94	1.36
9999	Refused	16,301	3.66	3.17
BLANK	Not asked or Missing	12,333		

Use the `table()` function to get a frequency table of the data in the `WEIGHT2` column in the code chunk below.

Given the information in the table above, what range of values would you say are in pounds in column `WEIGHT2`?

What range of values within the column `WEIGHT2` are recorded in kilograms?

```
table(brtri$WEIGHT2, useNA = "ifany")
```

```
##
##  50  56  58  59  60  61  62  65  67  68  70  71  72  74  75  76
##   1   1   1   1   3   1   1   3   4   1   3   1   3   3   3   1
##  77  78  80  81  82  83  84  85  86  87  88  89  90  91  92  93
##   2   4  13   1   2   1   3  12   5   3   4   7  44   2  11   7
##  94  95  96  97  98  99 100 101 102 103 104 105 106 107 108 109
##   9  44  19  20  32  20 207  14  39  36  37 193  40  61  75  31
## 110 111 112 113 114 115 116 117 118 119 120 121 122 123 124 125
```

```

## 394 26 120 70 77 398 88 86 147 64 902 55 123 130 82 763
## 126 127 128 129 130 131 132 133 134 135 136 137 138 139 140 141
## 114 161 182 102 1358 49 185 119 116 1135 139 147 253 94 1791 55
## 142 143 144 145 146 147 148 149 150 151 152 153 154 155 156 157
## 193 181 125 1153 124 186 247 107 2361 80 217 164 187 1079 161 170
## 158 159 160 161 162 163 164 165 166 167 168 169 170 171 172 173
## 262 92 2338 68 259 169 153 1524 87 155 302 114 2260 62 275 153
## 174 175 176 177 178 179 180 181 182 183 184 185 186 187 188 189
## 182 1543 137 93 288 105 2621 73 213 157 146 1331 116 146 131 140
## 190 191 192 193 194 195 196 197 198 199 200 201 202 203 204 205
## 1930 58 162 100 116 905 128 113 218 65 2649 45 98 80 106 622
## 206 207 208 209 210 211 212 213 214 215 216 217 218 219 220 221
## 77 92 123 77 1198 50 130 71 97 715 73 67 117 53 1390 30
## 222 223 224 225 226 227 228 229 230 231 232 233 234 235 236 237
## 86 48 61 655 42 36 69 29 1033 9 47 38 38 378 34 39
## 238 239 240 241 242 243 244 245 246 247 248 249 250 251 252 253
## 63 20 869 12 40 32 16 236 27 39 48 18 998 11 37 27
## 254 255 256 257 258 259 260 261 262 263 264 265 266 267 268 269
## 11 123 14 15 30 8 455 18 22 18 14 159 11 15 24 14
## 270 271 272 273 274 275 276 277 278 279 280 281 282 283 284 285
## 312 9 13 13 14 189 6 8 19 7 294 8 13 11 15 88
## 286 287 288 289 290 291 292 293 294 295 296 297 298 299 300 301
## 11 14 13 7 158 1 11 7 6 59 5 6 16 6 336 4
## 302 303 304 305 306 307 308 309 310 311 312 313 314 315 316 317
## 9 1 5 20 4 4 2 7 68 4 4 8 6 38 6 6
## 318 319 320 321 322 323 324 325 326 327 328 329 330 332 333 334
## 8 1 83 2 1 1 2 36 2 3 2 2 55 1 1 1
## 335 336 337 339 340 342 343 344 345 346 347 348 349 350 351 352
## 11 1 3 2 42 1 2 1 10 3 2 3 1 74 1 2
## 353 354 355 356 357 358 360 361 362 363 365 367 368 370 371 375
## 4 1 5 1 1 3 29 1 1 1 2 1 2 14 1 12
## 380 385 386 388 389 390 393 395 396 400 406 408 410 411 412 413
## 17 7 1 1 1 12 1 2 1 30 1 1 3 1 1 1
## 415 420 426 430 435 440 443 445 450 455 456 460 467 473 483 500
## 3 3 1 3 2 4 1 1 6 1 1 2 1 1 1 5
## 502 505 508 512 525 535 550 570 585 590 600 601 619 690 700 776
## 2 2 1 1 2 1 1 1 1 1 2 1 1 1 2 1
## 7777 9023 9024 9025 9030 9036 9037 9038 9039 9040 9042 9043 9045 9046 9047 9048
## 1262 2 1 2 2 1 2 3 2 2 4 2 7 4 4 2
## 9049 9050 9051 9052 9053 9054 9055 9056 9057 9058 9059 9060 9061 9062 9063 9064
## 3 19 3 6 10 16 25 15 10 14 9 64 5 22 22 13
## 9065 9066 9067 9068 9069 9070 9071 9072 9073 9074 9075 9076 9077 9078 9079 9080
## 44 9 16 31 16 62 7 18 16 16 42 15 9 26 12 57
## 9081 9082 9083 9084 9085 9086 9087 9088 9089 9090 9091 9092 9093 9095 9096 9098
## 4 15 13 5 26 6 11 9 4 22 1 5 4 12 2 4
## 9100 9102 9104 9105 9106 9107 9109 9110 9114 9115 9116 9118 9120 9122 9125 9127
## 8 2 2 3 1 3 1 5 2 2 2 1 5 1 2 1
## 9128 9129 9130 9135 9136 9140 9145 9150 9156 9160 9170 9173 9175 9178 9186 9204
## 1 1 1 1 1 1 1 1 1 3 1 1 2 1 1 1
## 9215 9238 9274 9285 9999 <NA>
## 1 1 1 1 2437 1921

```

21. Extracting weight: pounds

Create a new column called `WEIGHT1b` and populate it with `NA` values.

Extract the weight of each respondent recorded in pounds from the `WEIGHT2` column conditionally—only if

```
brtri$WEIGHT2>=50 & brtri$WEIGHT2<=776.
```

Use conditional methods with logical indexing and the `substr()` function to populate with the correct digits from the `WEIGHT2` column.

Remember that the last three digits of the `WEIGHT2` column represent respondent weight in kilograms if the leading digit is a '9'.

Convert values to pounds where necessary.

When subsetting columns, NA values are not allowed, so remember to include `!is.na(brtri$WEIGHT2)` in the logical indexing.

```
brtri$WEIGHT1b <- NA
brtri$WEIGHT1b[brtri$WEIGHT2 >= 50 & brtri$WEIGHT2 <=
  776 & !is.na(brtri$WEIGHT2)] <- brtri$WEIGHT2[brtri$WEIGHT2 >=
  50 & brtri$WEIGHT2 <= 776 & !is.na(brtri$WEIGHT2)]
brtri$WEIGHT1b[brtri$WEIGHT2 >= 9023 & brtri$WEIGHT2 <=
  9352 & !is.na(brtri$WEIGHT2)] <- as.numeric(substr(as.character(brtri$WEIGHT2[brtri$WEIGHT2 >=
  9023 & brtri$WEIGHT2 <= 9352 & !is.na(brtri$WEIGHT2)]),
  2, 4)) * 2.20462
```

22. Extracting weight: kilograms

Create a new column called `WEIGHTkg` and populate it with NA values.

Extract the weight of each respondent recorded in kilograms from the `WEIGHT2` column conditionally—only if `brtri$WEIGHT2>=9023 & brtri$WEIGHT2<=9352`.

Use conditional methods with logical indexing and the `substr()` function to populate with the correct digits from the `WEIGHT2` column.

Remember that the last three digits of the `WEIGHT2` column represent respondent height in centimeters if the leading digit is a '9'.

Convert values to kilograms where necessary.

When subsetting columns, NA values are not allowed, so remember to include `!is.na(brtri$WEIGHT2)` in the logical indexing.

```
brtri$WEIGHTkg <- NA
brtri$WEIGHTkg[brtri$WEIGHT2 >= 9023 & brtri$WEIGHT2 <=
  9352 & !is.na(brtri$WEIGHT2)] <- as.numeric(substr(as.character(brtri$WEIGHT2[brtri$WEIGHT2 >=
  9023 & brtri$WEIGHT2 <= 9352 & !is.na(brtri$WEIGHT2)]),
  2, 4))
brtri$WEIGHTkg[brtri$WEIGHT2 >= 50 & brtri$WEIGHT2 <=
  776 & !is.na(brtri$WEIGHT2)] <- round(brtri$WEIGHT2[brtri$WEIGHT2 >=
  50 & brtri$WEIGHT2 <= 776 & !is.na(brtri$WEIGHT2)] *
  0.453592, 2)
```

23. Compare our calculated `WEIGHTkg` column values to BRFSS weight column values

The Behavioral Risk Factor Surveillance Survey (BRFSS) data in the `brtri` dataframe have a codebook with the following information:

Label: Computed Weight in Kilograms

Section Name: Calculated Variables

Type of Variable: Num

Variable Name: WTKG3

Question: Reported weight in kilograms

Value	Value Label	Frequency	Percentage	Weighted Percentage
2300-29500	Weight in kilograms [2 implied decimal places] Notes: 0001 <= WEIGHT2 <= 650 or 9023 <= WEIGHT2 <= 9295 (non-metric WEIGHT2 value divided by 2.2046)	421,278	100.00	100.00
BLANK	Don't know/Refused/Not asked or Missing Notes: WEIGHT2 = 7777 or 9999 or not in accepted values or WEIGHT2 = Missing	36,392		

Use the `head()` function to view the first rows of the `WEIGHTlb`, `WEIGHTkg`, and `WTKG3` columns in the `brtri` dataframe. Based on what you see, how do the values of the column `WEIGHTkg` compare to the column `WTKG3`?

```
head(brtri[, c("WEIGHTlb", "WEIGHTkg", "WTKG3")])
```

```
##  WEIGHTlb WEIGHTkg WTKG3
## 1      205     92.99  9299
## 2       NA       NA    NA
## 3      175     79.38  7938
## 4      230    104.33 10433
## 5      240    108.86 10886
## 6      132     59.87  5987
```

24. Calculate BMI for each respondent from `WEIGHTkg` and `HEIGHTtotm`

From respondent weight in kilograms and height in meters, create a new column `BMIall` by calculating BMI in kilograms per meters squared.

```
brtri$BMIall <- brtri$WEIGHTkg/(brtri$HEIGHTtotm)^2
```

25. Compare our calculated `BMIall` column values to BRFSS BMI column values

The Behavioral Risk Factor Surveillance Survey (BRFSS) data in the `brtri` dataframe have a codebook with the following information:

Label: Computed body mass index
Section Name: Calculated Variables

Type of Variable: Num
Variable Name: BMI5

Question: Body Mass Index (BMI)

Value	Value Label	Frequency	Percentage	Weighted Percentage
1-9999	1 or greater Notes: WTKG3/(HTM4*HTM4) (Has 2 implied decimal places)	414,633	100.00	100.00

Value	Value Label	Frequency	Percentage	Weighted Percentage
BLANK	Don't know/Refused/Missing Notes: WTKG3 = 777 or 999 or HTM4 = 777 or 999			

Use the `head()` function to view the first rows of the `BMIa11` and `BMI5` columns in the `brtri` dataframe. Based on what you see, how do the values of the column `BMIa11` compare to the column `BMI5`?

```
head(brtri[, c("BMIa11", "BMI5")])
```

```
##      BMIa11 BMI5
## 1 34.99944 3519
## 2      NA    NA
## 3 27.46713 2741
## 4 36.10035 3602
## 5 40.97256 4120
## 6 25.91326 2578
```